OFFICE POC • BENCHMARK & ARCHITECTURE REFERENCE

Document Parser Selection & Smart Routing Guide

Evidence-based parser mapping across 96 test documents (62 Baseline + 34 Stress Tests) scored against hand-verified Ground Truth (0–5 scale).

96 Docs	786+ Runs	3 Core Free	0.0s – 13s
62 Baseline + 34 Stress Tests	8 Ground-Truth Metrics	PyMuPDF, ODL, PyMuPDF4LLM	Fast Route Latency Range

1. Master Parser Matrix — All 11 Document Types

Quick Reference

No single parser wins across every file format. Below is the definitive routing map showing the best overall parser and the best **Free & Fast** choice (when avoiding slow Docling runs or paid API calls).

Document Type	Best Overall Parser	Best Free & Fast Route	Score & Speed	Primary Reason
1. Digital PDF (Simple Text)	`pymupdf`	`pymupdf`	**4.94** (0.0s)	Instant direct text-layer extraction; highest accuracy.
2. Digital PDF (Long 20–45p Reports/Contracts)	`pymupdf4llm`	`pymupdf4llm`	**4.99–5.00** (12–22s)	Preserves nested clause numbering & Markdown headings.
3. Digital PDF (Standard Tables, POs, 100p Ledgers)	`opendataloader`	`opendataloader`	**4.45–5.00** (2.7–13s)	Matches Docling quality at 10× speed; 5.00 on 100p ledger.
4. Digital PDF (Merged Headers & Side-by-Side Tables)	`pymupdf4llm` / `docling`	`pymupdf4llm`	**4.62–5.00** (2–6s)	ODL fails side-by-side tables (0.00 table); 4LLM keeps grid.
5. Digital PDF with Charts (Values Matter)	`smart_pdf` (Paid)	`pymupdf4llm` (Text only)	**3.88–4.53** (0.5–9s)	Only Vision LLM reads numbers inside chart bars/lines (5.00).
6. Digital PDF with Photos / Flowcharts Only	`pymupdf4llm`	`pymupdf4llm`	**4.40–4.94** (7.4s)	Vision LLM adds unwanted photo descriptions; 4LLM stays clean.
7. Mixed PDF (Digital Text + Scanned Pages)	`pymupdf4llm` / `docling`	`pymupdf4llm`	**4.42** (8.0s)	Docling takes 42.5s (4.51); 4LLM finishes in 8.0s (4.42).
8. Scanned PDFs & Mobile Camera Photos	`docling`	`pymupdf4llm` (Clean scans)	**3.47–4.67** (11–13s)	4LLM scores 4.49 on clean scans; Docling (4.67) needed for tilted photos.
9. Standalone Images (.png, .jpg)	`docling`	`easyocr` / `ocr`	**4.85** (12.6s)	Docling leads (4.85); EasyOCR (4.68, 2.6s) & OCR (4.51, 0.7s) are fast.
10. Word Documents (.docx)	`unstructured`	`unstructured`	**4.20–4.93** (0.4s)	Retains running page headers & nested lists that native docx drops

2. Deep Dive: PDF Tables, Complex Layouts & Scans

Why One Parser Isn't Enough

Colleagues often ask why we cannot use **OpenDataLoader** or **Tesseract OCR (hybrid_pdf)** for every PDF. The ground-truth benchmark proves that each table and scan parser has strict structural boundaries:

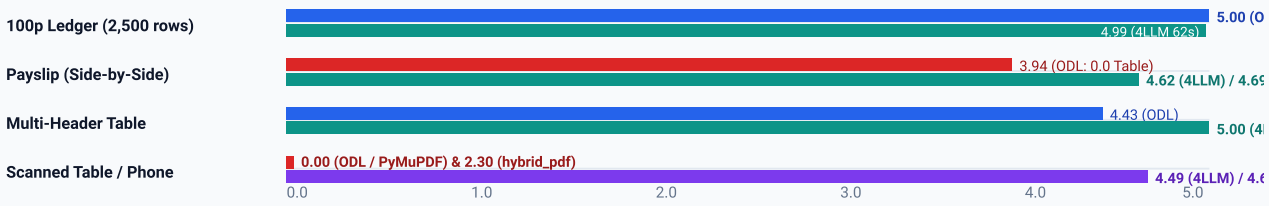


Figure 1: Measured Ground-Truth Scores (0–5) across stress-test files showing where OpenDataLoader excels vs. where PyMuPDF4LLM and Docling rescue failures.

3. Specific Business File Subtypes — Which Parser Wins & Why?

Stress-Test Evidence

Business File Subtype	Winning Parser	Winner Score	Why Others Failed on This Exact Document
Standard Invoices, POs & Bills of Lading	`opendataloader`	**4.56 – 4.96** (1.5s – 3.0s)	Plain `pymupdf` repeats merged cells across columns (**4.11** on Invoice); `markitdown` & `unstructured` lose table gridlines completely.
100-Page Transaction Ledger (2,500 rows)	`opendataloader`	**5.00** (13s)	`pymupdf4llm` scored **4.99** but took 62s; plain `pymupdf` took 294s (**3.01**); `docling` took 86s (**4.97**). ODL is 5× faster.
Multi-Row Merged Header Tables (Clinical / Financial)	`pymupdf4llm`	**4.86 – 5.00** (6s – 20s)	`opendataloader` broke header hierarchy (Structure score dropped to **2.50**, overall **4.43**). `smart_pdf` mistook table gridlines for an image (**2.96**).
Side-by-Side Tables (Employee Payslip)	`docling` / `pymupdf4llm`	**4.69 / 4.62** (2s for 4LLM)	`opendataloader` merged earnings & deductions across columns, scoring **0.00** on Table metric (**3.94** overall).
Nested Tables & Multi-Column Forms (Policy / Lab)	`pymupdf4llm` / `opendataloader`	**4.55 – 4.77** (1.5s – 3.0s)	`hybrid_pdf` and `smart_pdf` rely on plain PyMuPDF for digital text, which scrambles inner table cells (scoring only **2.92**).
60-Page Multi-Column Annual Report (10 Charts)	`pymupdf4llm`	**4.18** (52s)	`docling` hit the **600-second timeout** and failed; `opendataloader` trailed at **3.91** (16s); `unstructured` scored **3.26** (154s).
Clean Scanned Financial Statement (PDF Scan)	`pymupdf4llm` / `docling`	**4.49 / 4.41** (11s vs 29s)	`opendataloader` & `pymupdf` returned **0.00 (empty)**. `hybrid_pdf` (Tesseract) destroyed the table into flat lines (**2.36**, Table **0.00**).
Tilted / Noisy Mobile Camera Photo (Scanned PO)	`docling`	**4.67** (13s)	`hybrid_pdf` / Tesseract scored **2.30** (Table **0.00**); `pymupdf4llm` scored **3.58** (Table **0.50**). Only Docling's layout model recovered the tilted table.

Why Did `hybrid_pdf`, `smart_pdf`, and Frameworks (Haystack/LangChain) Fail on Tables & Scans?

- **`hybrid_pdf` (PyMuPDF + Tesseract OCR):** Tesseract only reads characters line-by-line; it has zero table structure detection. On scanned tables, its Table score is **0.00** (overall **2.30–2.53**). Use `pymupdf4llm` or `docling` for scans instead.
- **`smart_pdf` (Hybrid + Azure OpenAI Vision):** Essential *only* when a PDF has bar/line/pie charts whose numbers must be read (scoring **4.83–5.00** on Charts). On complex tables without charts, its figure detector mistakes table borders for pictures and duplicates numbers (dropping score from **5.00 to 2.96**).

4. Non-PDF Formats: Word, PowerPoint, Excel, CSV & Images

Office & Media

For non-PDF documents, routing to heavy PDF engines wastes time and lowers quality. Below are the measured winners across Office, text, and image files:

File Format	Best Parser	Measured Score	Selection Rationale & Rejected Alternatives
Word (.docx) SOPs, Manuals, Memos	`unstructured` (or `docling`)	**4.20 – 4.93** (0.4s – 0.8s)	Native `docx` reader (**4.09–4.33**) drops running page headers (missing Document IDs) and flattens nested numbered lists. `unstructured` preserves both at 0.4s.
PowerPoint (.pptx) Standard Text/Table Decks	`pptx` (Native)	**4.86** (0.0s)	Instant and lossless on text/table slides. Beats `docling` (**4.58**) and `unstructured` (**4.57**).
PowerPoint (.pptx) Decks with Native Charts	`docling` (or `markitdown`)	**4.50 – 4.79** (0.3s – 1.3s)	Native `pptx` reader and `unstructured` skip chart objects entirely (Chart score **0.00**, overall **2.76**). `docling` extracts embedded chart series (**5.00** chart score in 1.3s).
Excel (.xlsx) Multi-Sheet & Financial	`spreadsheet` / `markitdown`	**4.33 – 4.99** (0.0s – 0.8s)	Native `spreadsheet` reader scores **4.99** on 5-sheet workbooks in 0.1s. On budget sheets with embedded chart objects, `markitdown` (**4.31**) & `unstructured` (**4.45**) edge it out.
CSV (.csv) & Plain Text (.txt, .md)	`spreadsheet` / `text`	**5.00** (0.0s)	Perfect **5.00** in 0.0s. Sending plain text to `unstructured` drops score to **2.08** due to chunking distortion.
Images (.png, .jpg) Screenshots & Scans	`docling` (Fast: `easyocr`)	**4.85** (12.6s) **4.68** (2.6s)	`docling` preserves layout & tables inside images (**4.85**). If speed is critical, `easyocr` (**4.68** in 2.6s) or `ocr` (**4.51** in 0.7s) are strong fast alternatives.

5. How Result-Based Smart Routing Works in Production

Architecture



Figure 2: Automated Smart Extract routing pipeline based on `analyzer.py` and `measured_scores.json`.

Why "By Document" Beats "By Part" Mode

In benchmark testing, routing the whole file (**By Document: 4.32–4.59 avg**) consistently outperformed splitting pages into pieces (**By Part: 4.05–4.26 avg**). "By Part" hands table regions to plain PyMuPDF, which duplicates merged cells (dropping Lab Report from **4.55 to 3.26**).

Tie-Breaker & Cost Control Rule

Any parser within **0.1 points** of the top score is treated as tied for accuracy, and the router picks the **fastest parser**. Paid parsers (`smart_pdf`, `azure_di`) are only triggered when explicitly enabled and when data charts require Vision extraction.

3 Minor Code Fixes Recommended Next

1) In `figures.py`, ignore figure boxes that overlap detected tables. 2) In `learned.py`, route `.docx` to

Final Verdict for Stakeholders

By pairing **PyMuPDF** (text), **OpenDataLoader** (standard tables/ledgers), and **PyMuPDF4LLM**